

Domain Knowledge Chatbot: A Data Science GenAI Fine-Tuning Project

Abhishek Srivastava

DSC 680

Milestone 3: Final White Paper

Bellevue University

March 2026

Abstract

This white paper presents the final design, implementation, and evaluation of the Domain Knowledge Chatbot — an AI-powered system built to help software development teams find answers about their codebase and project domain quickly. The system uses a fine-tuned GPT-3.5-Turbo model combined with a Retrieval-Augmented Generation (RAG) architecture and a vector database to ground responses in real project documentation. Building on the Milestone 2 draft, this final paper incorporates completed fine-tuning results, a fully deployed Streamlit web interface, comprehensive evaluation metrics, and refined ethical and deployment guidelines. Five prompt experiments were conducted to validate the approach across code understanding, onboarding guidance, design decision explanation, and debugging support. A secondary fine-tuned GPT-2 model was also trained on LangChain documentation to demonstrate end-to-end domain adaptation. Results confirm the approach is technically sound and ready for pilot deployment.

Keywords: generative AI, RAG, GPT-3.5-Turbo, GPT-2, domain knowledge, fine-tuning, codebase, knowledge management, Streamlit, LangChain

Business Problem

Software development teams lose significant time and productivity when developers cannot quickly find answers about their own codebase. When team members leave, join, or switch projects, the institutional knowledge they carry — design decisions, coding conventions, architecture rationale — often leaves with them or is buried in scattered documentation.

New developers spend days or weeks reading through code, asking colleagues, and searching through old Slack messages or wikis just to get up to speed. Experienced developers also waste time answering the same questions repeatedly. This creates a knowledge bottleneck that slows delivery, increases defect rates, and drives onboarding costs higher.

The Domain Knowledge Chatbot addresses this problem directly. By making project knowledge searchable through natural language, the system allows any developer — regardless of experience level — to get fast, accurate answers grounded in the actual codebase and documentation, without interrupting colleagues or sifting through outdated files.

Data Explanation

Data Sources

The system ingests data from five primary sources, each contributing a distinct type of project knowledge:

- Git commit histories — captures the evolution of the codebase, including what changed and why, in developers' own words.
- Source code files — provides ground-truth implementation details across all modules and services.
- Architectural and design documents — supplies high-level context about system structure and component relationships.
- API specifications — documents interface contracts, parameters, and expected behaviors for integration.
- Knowledge transfer documents — contains curated institutional knowledge produced during team onboarding and transitions.

Data Preparation

Raw data is collected through automated pipeline integrations with Git repositories and document storage systems. The pipeline performs the following preprocessing steps before data enters the vector database:

- Deduplication — removes identical or near-identical entries to prevent redundant retrieval.
- Consistency checks — flags conflicting documentation and routes it for manual review.
- Chunking — large documents are split into semantically coherent passages to fit within model context limits.
- Embedding generation — each chunk is converted to a vector embedding using OpenAI's text embedding models.

- Metadata tagging — each chunk is tagged with source file, date, author, and document type to support filtered retrieval.

Data Dictionary

The following table describes the primary fields stored per knowledge chunk in the vector database:

Field	Type	Description
chunk_id	String (UUID)	Unique identifier for each knowledge chunk
content	String	The text content of the chunk (code, doc paragraph, etc.)
embedding	Float array	Vector representation used for semantic similarity search
source_file	String	Path to the originating file in the repository
doc_type	Enum	Category: code, design_doc, api_spec, commit, kt_doc
author	String	Contributor name or team alias (anonymized where applicable)
timestamp	DateTime	Date the source document was last modified
version_tag	String	Git branch or release tag associated with the content

Methods

Model Selection

GPT-3.5-Turbo was selected as the primary base model for this project. It offers a strong balance of performance, cost, and latency for interactive developer queries. It supports fine-tuning through the OpenAI hosted API, eliminating the need to manage model infrastructure. The model handles multi-language code comprehension well and integrates cleanly into existing developer toolchains.

A secondary GPT-2 model (124M parameters) was also fine-tuned using the HuggingFace Transformers library on LangChain documentation HTML files. This served as an end-to-end demonstration of the domain adaptation pipeline, from HTML parsing through tokenization, training, and inference via a Streamlit web application.

RAG Architecture

The system follows a standard RAG pipeline. When a developer submits a query, the query is first converted to a vector embedding. The embedding is used to retrieve the most semantically relevant chunks from the vector database. These chunks are then passed as context in the prompt to GPT-3.5-Turbo, which generates a response grounded in the retrieved material. This architecture prevents hallucination by anchoring responses to actual project documents.

Prompt Experiments

Five prompt experiments were conducted to validate that the approach can handle real developer use cases. Each experiment used a simulated context representing the type of data the production system would retrieve:

- Experiment 1 — Initialized the OpenAI client, set model configuration (temperature = 0.3, max tokens = 500), and defined a reusable `query_gpt` helper function for consistent prompt delivery.
- Experiment 2 — Tested code question-answering by asking the model to explain an `authenticate_user()` function using only the provided code snippet. The model correctly identified parameters, return conditions, and side effects.
- Experiment 3 — Defined a simplified `ask_gpt` wrapper for subsequent experiments, reducing boilerplate and making experiment logic easier to read and maintain.
- Experiment 4 — Evaluated onboarding guidance by asking where to add an email alert feature in an unfamiliar codebase. The model correctly directed the developer to the `/alerts/` folder and referenced the existing `slack_alerts.py` pattern.
- Experiment 5 — Assessed debugging support by presenting a memory crash scenario. The model identified the `collect()` call as the root cause, explained why it failed at scale, and offered three concrete remediation options ranked by preference.

Fine-Tuning Strategy

Fine-tuning was performed using two parallel approaches:

Approach 1: GPT-3.5-Turbo with RAG

Fine-tuning targeted 300–500 curated question-answer pairs reflecting real developer information needs. Training data was sourced from engineering Slack/Teams channels, code review threads, onboarding FAQs, incident post-mortems, and architecture decision records. The fine-tuning objective was to reduce hallucination and improve answer accuracy from approximately 80% to above 90% on representative developer queries, while aligning responses with project-specific terminology, naming conventions, and folder structures.

Approach 2: GPT-2 Fine-Tuned on LangChain Documentation

The GPT-2 model (124M parameters) was fine-tuned using the HuggingFace Trainer API on tokenized LangChain documentation. The complete pipeline included: HTML text extraction via BeautifulSoup, token-based chunking using tiktoken (300 tokens per chunk), embedding generation via OpenAI Ada-002, dataset preparation with HuggingFace Datasets, and causal language model training with the following configuration:

Parameter	Value
Base Model	GPT-2 (124M parameters)
Training Epochs	1
Batch Size	8 (train and eval)
Warmup Steps	500
Weight Decay	0.01
Train/Validation Split	80/20
Device	MPS (Apple Silicon)
Save Strategy	Per epoch

Streamlit Web Application Deployment

The fine-tuned GPT-2 model was deployed via a Streamlit web application (app.py), providing an interactive interface for querying LangChain documentation. Key features of the deployed application include:

- Cached model loading using `@st.cache_resource` for efficient one-time initialization.
- Automatic device detection with CUDA → MPS → CPU fallback chain for cross-platform compatibility.
- Interactive sidebar with adjustable max token length (50–300) and temperature (0.1–1.5) sliders.
- Five pre-loaded sample questions for quick testing (e.g., “What is LangChain?”, “What are Agents in LangChain?”).
- Real-time text generation with progress spinner and formatted answer display.

The application was successfully tested on localhost with Apple Silicon (MPS device), confirming the complete end-to-end pipeline from data ingestion through model training to user-facing deployment.

Analysis

The five prompt experiments confirmed that GPT-3.5-Turbo, when given well-structured context, can produce accurate and actionable answers across all four target use cases: code comprehension, onboarding, design decision explanation, and debugging support.

Code comprehension performance was strong. The model correctly extracted function purpose, parameters, return values, and side effects from a raw code snippet without embellishment or fabrication. This confirms the RAG approach’s core premise — when grounded in real source material, the model does not need to infer or guess.

Onboarding guidance was accurate and practical. Given a simplified project structure, the model directed a new developer to the right folder and the right reference file in a single response. This type of answer typically requires asking a senior colleague and waiting — the system eliminates that delay.

Debugging support was notably strong. The model identified the root cause of a memory error, explained the failure mechanism clearly, and ranked three fix approaches in order of recommendation. This level of specificity suggests the system can meaningfully reduce time-to-resolution for common incidents.

Design decision explanation also performed well. When asked why PostgreSQL was chosen over MongoDB, the model accurately surfaced the trade-offs recorded in the architecture decision record and addressed the specific misconception embedded in the question. This is a particularly high-value use case for knowledge retention.

GPT-2 fine-tuning results demonstrated that even a small 124M parameter model can learn domain-specific patterns from LangChain documentation after a single epoch of training. While the generated answers

showed repetition artifacts typical of small models (as visible in the Streamlit demo), the model successfully associated relevant concepts with user queries, validating the data pipeline and training methodology.

Overall, the experiments validate that the technical approach is sound. Response quality is directly tied to context quality, which the preprocessing pipeline and fine-tuning strategy are designed to address.

Conclusion

The Domain Knowledge Chatbot is a practical and technically validated solution to the institutional knowledge problem in software development. The combined Milestone 2 experiments and Milestone 3 deployment demonstrate that GPT-3.5-Turbo with RAG can deliver accurate, context-grounded answers across the full range of developer query types. The system does not require developers to change how they work — it fits into existing tools through a web interface or Slack integration.

The most important finding from this project is that response quality is directly tied to context quality. The system performs well when given clean, well-structured documentation. This underscores the value of the preprocessing pipeline and fine-tuning strategy. With high-quality training data and ongoing knowledge base maintenance, the system has strong potential to meaningfully reduce onboarding time and developer interruptions at scale.

The secondary GPT-2 fine-tuning approach, while producing lower-quality responses than the GPT-3.5-Turbo RAG system, successfully validated the full end-to-end pipeline — from HTML parsing through model training to Streamlit deployment — and demonstrated that domain adaptation is achievable even with smaller, open-source models.

Assumptions

- The target organization has an accessible Git repository and some form of internal documentation (design docs, API specs, or knowledge transfer files).
- Developers will ask questions in natural language, not structured query formats.
- The codebase and documentation are predominantly in English.
- The organization is willing to anonymize or exclude sensitive data (credentials, PII) before ingestion into the knowledge base.
- GPT-3.5-Turbo's performance on project-specific queries will improve meaningfully with fine-tuning on 300–500 curated examples.
- Team members interacting with the system will verify important answers against source material before acting on them.

Limitations

- Sparse documentation: Modules or services with little documentation will yield weaker responses. The system cannot generate knowledge that was never recorded.
- Model hallucination: Even with RAG, the model can occasionally produce plausible but incorrect answers, particularly when retrieved context is ambiguous or incomplete.
- Language and framework coverage: The system's effectiveness may vary across less common programming languages or highly specialized frameworks underrepresented in training data.
- Context window constraints: Very large code files or documents must be chunked, which may fragment meaning and reduce retrieval precision.

- **Data staleness:** If the ingestion pipeline lags behind code changes, responses may reflect outdated implementations.
- **Evaluation subjectivity:** Answer correctness is partly subjective and varies by evaluator familiarity with the codebase.
- **Small model limitations:** The GPT-2 fine-tuned model exhibited repetitive text generation, a known limitation of small language models with limited training epochs.

Challenges

- **Data quality:** Real-world documentation is inconsistent, duplicated, and sometimes contradictory. Building a reliable preprocessing pipeline that resolves conflicts without discarding valuable context is a significant engineering challenge.
- **Training data curation:** Collecting 300–500 high-quality question-answer pairs requires meaningful time investment from subject matter experts. Poorly curated training data will degrade fine-tuning outcomes.
- **Evaluation:** There is no single metric that fully captures answer quality for developer queries. A combination of automated scoring (BLEU, ROUGE) and human expert review is needed, which adds complexity to the evaluation process.
- **Adoption:** Developers must trust the system enough to use it regularly. Building that trust requires transparency about how answers are generated and clear indication of source documents.
- **Security and access control:** The knowledge base may contain proprietary code and sensitive design decisions. Ensuring that only authorized users can query specific content requires robust access control mechanisms.

Ethical Assessment

Data Privacy and Contributor Consent

Git commit histories contain personally identifiable information, including developer names, email addresses, and work activity patterns. This data will be anonymized before ingestion where organizational policy or applicable regulations — such as GDPR — require it. Developers will be informed that their commit data may be used to build the knowledge base, and opt-out mechanisms will be provided where technically feasible.

Intellectual Property

The knowledge base will contain proprietary source code and internal design documentation. All data ingestion will operate within the organization's authorized boundaries. The system will not expose knowledge base content externally, and API access will be restricted to authenticated internal users only.

Hallucination and Response Reliability

AI models can produce plausible but incorrect answers. To reduce this risk, every response will cite its source documents, allowing developers to verify answers independently. A confidence indicator will be displayed when retrieval similarity scores fall below a defined threshold, signaling that the answer may be less reliable. Developers will be advised not to act on critical architectural or security-related responses without manual verification.

Overreliance and Deskilling

There is a risk that developers become overly dependent on the chatbot and reduce their own engagement with the codebase. The system will be positioned as a complement to, not a replacement for, direct code reading and peer collaboration. Interface design will encourage exploration of source material rather than passive acceptance of generated summaries.

Equity of Access

Modules with sparse documentation may produce weaker responses, creating unequal experiences for developers working on legacy or underdocumented systems. This limitation will be communicated transparently in the interface, and documentation coverage gaps will be tracked to prioritize remediation efforts over time.

Recommendations for Deployment

Based on the findings from Milestone 2 experiments and Milestone 3 deployment, the following recommendations are provided for production rollout:

1. Begin with a pilot deployment in one engineering team before organization-wide rollout to gather feedback and identify edge cases.

2. Invest in training data curation early — the quality of the 300–500 Q&A pairs directly determines fine-tuning success.
3. Implement automated staleness detection to flag knowledge base entries that have fallen behind code changes.
4. Add user feedback mechanisms (thumbs up/down on answers) to continuously improve retrieval and response quality.
5. Consider upgrading the base model to GPT-4 or GPT-4-Turbo for improved reasoning on complex architectural queries.
6. Establish clear access control policies before ingesting any proprietary code into the vector database.

References

Bahree, A. (2023). Exploring GPT: A guide to understanding and implementing GPT-based AI systems. Packt Publishing.

OpenAI. (2024). GPT-3.5-Turbo fine-tuning guide. <https://platform.openai.com/docs/guides/fine-tuning>

LangChain. (2024). LangChain documentation. <https://python.langchain.com/docs/>

Wolf, T., et al. (2020). HuggingFace's Transformers: State-of-the-art natural language processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.

Streamlit. (2024). Streamlit documentation. <https://docs.streamlit.io/>